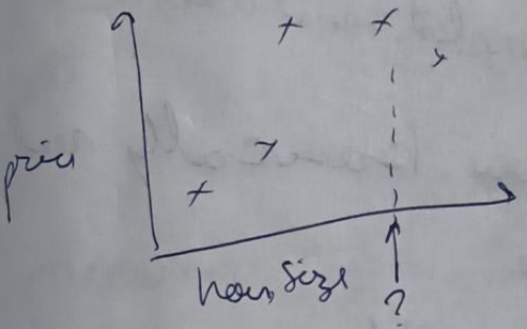
Lecture 1

for project refference: cs229.stanford.edu

Supervised Learning → we are given data-set of form (x, y) and we need to find a map ⇒ is/isn't supervised learning only x and we need to clusterize etc

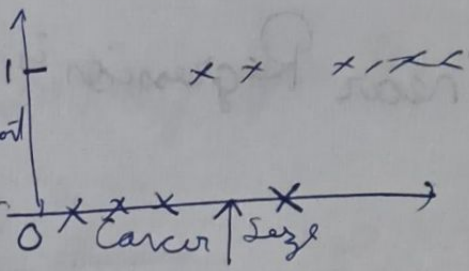
Regression Problem

→ y axis has continous value here graph also continous

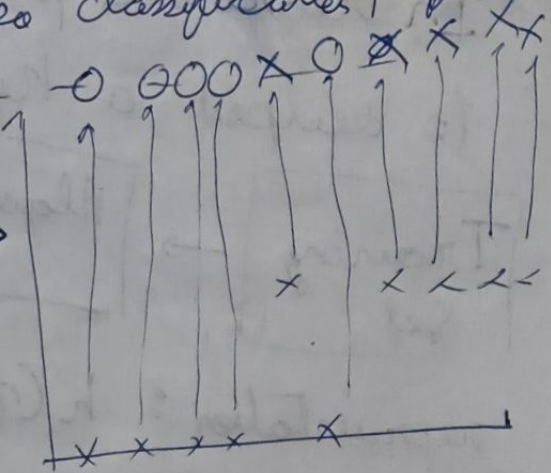


Classificates

→ not continous

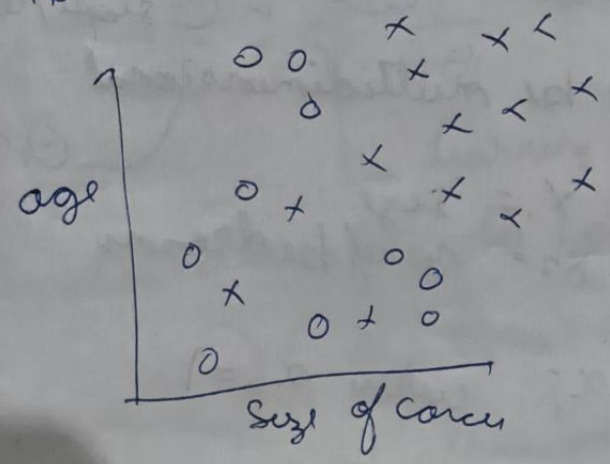


even if 5 type of cancer, its also classificates problem



this representation is imp → because input is usually not 1D

ex:



Support Vector Machine → can store ∞ dimension input

Unsupervised Learning → We only get x and have to figure out pattern & grouping
like clustering → Google News
→ mostly about unlabeled (no Y) data

Lecture 2

Linear Regression is : (Supervised learning + Regression Problem) Simplest one of that

In regression problem we basically need to output a hypothesis

Training Set → Algorithm → H

representation : $h(x) = \theta_0 + \theta_1 x$ (intercept/bias, slope/weight)

~~x=∞~~ x can be multidimensional $\theta_1 x_1 + \theta_2 x_2 \dots$

like x_1 = size
x_2 = no. of bedrooms

$$h(x) = \sum_{j=0}^{n} \theta_j x_j \quad \text{where } x_0 = 1$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \end{bmatrix}, \quad x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \end{bmatrix} \qquad h_\theta(x) \equiv h(x)$$

↓ Parameters

n = no. of training examples / no. of rows in table

x = inputs / features
y = output / target variable
$(x^{(i)}, y^{(i)}) \rightarrow i^{th}$ training example (not power, just weird notation for index)

n = no. of features

formally ⇒ Choose θ s.t. $h(x) \approx y$ for the training

~~x~~ → minimize $(h_\theta(x) - y)$

minimize $\frac{1}{2} \left(\sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right) \right)^2 \equiv J(\theta)$

↳ convention

to minimize $J(\theta)$:

Gradient Descent

$\theta_j := \theta_j - \frac{\partial J(\theta_j)}{\partial \theta_j}$ (learning rate, practice : set to 0.01)

same as $a = a+1$ in C++, not valid in math, ∴ $a \neq a+1$

for derivation derivation, do col. for only one training set element

$$\frac{\partial J(\theta_j)}{\partial \theta} = \left(h_{\theta_j}(x) - y \right) \left(\frac{\partial}{\partial \theta_j} [\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 \cdots \theta_n x_n - y] \right)$$

for θ_j all 0 except $\theta_j x_j \rightarrow x_j$

$$\Rightarrow \frac{\partial J}{\partial \theta} = (h_\theta(x) - y) \cdot x_j$$

$$\Rightarrow \theta_j := \theta_j - (\alpha)(x_j)(h_\theta(x) - y)\, x_j$$

$$\theta_j := \theta_j - (\alpha)\left\{\sum_{i=1}^{m}\right\}(h_\theta(x^i) - y^i)\cdot x_j$$

\# if α is too large → overshoot post minima
if α is too small → slow algo :: more iterations

aka Batch Gradient descent
m = size of our batch

Stochastic Gradient Descent

Repeat { for $i = 1$ to n { $\theta_i := \theta_i - \alpha(h_\theta(x^i) - y^i)\cdot x^i$ } }

Batch Gradient Descent converges, stochastic gradient descent doesn't

Stochastic is better for large dataset

Normal Equation

works only for linear regression. Finds out the optimal value and directly jumps to it

notation: $\nabla_\theta J(\theta)$ → derivative of $J(\theta)$ wrt θ

$$\begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \end{bmatrix}$$

$\theta \in \mathbb{R}^{n+1}$ :: 0

Ex of notation: if $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \frac{\partial f}{\partial A_{12}} \\ \frac{\partial f}{\partial A_{21}} & \frac{\partial f}{\partial A_{22}} \end{bmatrix}$$

some properties

if we define $f(A) = \mathrm{tr}(AB)$ (B some fixed matrix)
then $\nabla_A f(A) = B^T$

$\mathrm{tr}(AB) = \mathrm{tr}(BA)$
$\mathrm{tr}(ABC) = \mathrm{tr}(CAB)$

$$\nabla_A \mathrm{tr}(AA^TC) = CA + C^TA$$

(similar to $\frac{\partial A^2 C}{\partial A} = 2AC$)

we define a matrix X (x^i is a training example)

$$X = \begin{bmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ (x^{(3)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix} \quad \text{now} \quad X\theta = \begin{bmatrix} (x^{(1)})^T \\ (x^{(2)})^T \\ (x^{(3)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

$$= \begin{bmatrix} (x^{(1)})^T\theta \\ (x^{(2)})^T\theta \\ \vdots \\ (x^{(m)})^T\theta \end{bmatrix} = \begin{bmatrix} h_\theta(x^1) \\ h_\theta(x^2) \\ \vdots \\ h_\theta(x^m) \end{bmatrix}$$

we also difin $\vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$

now we can rewrite $J(\theta)$ as

$= \frac{1}{2}(X\theta - y)^T(X\theta - y)$

$X\theta - y = \begin{bmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ \vdots \\ h_\theta(x^{(m)}) - y^{(m)} \end{bmatrix}$

and

$z^T \cdot z = \sum_i z_i^2$

$\Rightarrow \nabla_\theta J(\theta) = \nabla_\theta \frac{1}{2}(X\theta - y)^T(X\theta - y)$

$= \frac{1}{2}\nabla_\theta (\theta^T X^T - y^T)(X\theta - y)$

$= \frac{1}{2}\nabla_\theta \left[\theta^T X^T X\theta - \theta^T X^T y - y^T X\theta + y^T y\right]$

$= \frac{1}{2}\left[X^T X\theta + X^T X\theta - X^T y - X^T y\right]$

$= X^T X\theta - X^T y \quad \text{set} = \vec{0}$

$\Rightarrow X^T X\theta = X^T y \longrightarrow$ Normal Eqn

$\Rightarrow \theta_{optimal} = (X^T X)^{-1} X^T y$

use pseudo inverse if X is non invertable

Lecture 3

Intro

Classification algo → logestic regression

locally weighted regression → modification of linear regression to fit non linear graphs

for straight line

we use $\theta_0 + \theta_1 x_1$

but maybe a strt line is not good enough

for quadratic: $\theta_0 + \theta_1 x + \theta_2 x^2$

or maybe $\theta_0 + \theta_1 x + \theta_2 \sqrt{x} + \theta_3 \log(x)$

↳ for better idea we use (locally weighted regression)

locally weighted regression is non-parametric

ie- Amount of data/parameters grow (likely) with size of data

is LR fit θ to minimize

$$\frac{1}{2}\sum_i \left(y^{(i)} - \theta^T x^{(i)}\right)^2$$

Return $\theta^T x$

is Locally WR

Fit θ to minimize

$$\sum_{i=1}^{n} w^{(i)} \left(y^{(i)} - \theta^T x^{(i)}\right)^2$$

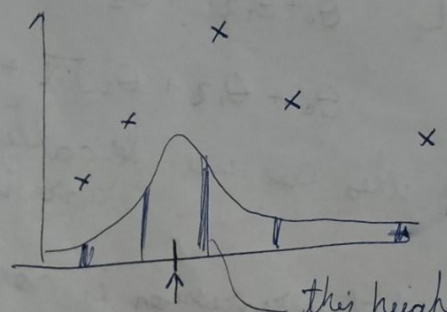
Where $w^{(i)}$ is a weight function

prolly $w^{(i)} = \exp\left(\frac{-(x^{(i)} - x)^2}{2\tau^2}\right)$ ← changes width of bell curve

if $|x^{(i)} - x|$ is small $w^{(i)} \approx 1$)

if $|x^{(i)} - x|$ is larg $w^{(i)} \approx 0$) → more emphasis to closer data pts

Ex:



Note: $\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$

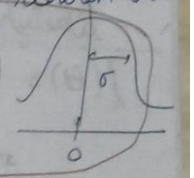
$\sim$ is distributed as; $\mathcal{N}$ normal distribution / gaussian distribution

Why least square is loss func^

Assume: $y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$

$\varepsilon^{(i)}$ ↑ "error" + random noise

$$\Rightarrow p(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(\varepsilon^{(i)})^2}{2\sigma^2}\right)$$



Assumption: The ε ie noise are IID (independently & identically distributed)

which is not correct

ex: if price of one house on strip is unusually high, other house on same street would prolly also have higher than usual pricing here not really independent

This implies

$$p(y^{(i)} \mid x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

parametrized by θ; equal? $= \varepsilon^{(i)}$

ie $y^{(i)} \mid x^{(i)}; \theta \sim \mathcal{N}(\theta^T x^{(i)}, \sigma^2)$

Notation: $\mathcal{L}(\theta)$ → likelyhood of parameter theta

$$L(\theta) = p(\vec{y}|x;\theta) = \prod_{i=1}^{m} p(y^{(i)}|x^{(i)};\theta)$$

$$= \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

probability of data
likelyhood of parameters

$$l(\theta) = \log(L(\theta))$$

$$= \sum_{i=1}^{m} \left[\log \frac{1}{\sqrt{2\pi}\sigma} + \log(\exp(\cdot)) \right]$$

$$\text{constant} = \left(m \log\left(\frac{1}{\sqrt{2\pi}\sigma}\right) \right) + \sum_{i=1}^{m} \left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2} \right)$$

constant

So choose θ to maximize $L(\theta)$

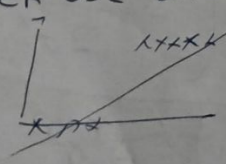
So minimize $\sum_{i=1}^{m} (y^{(i)} - \theta^T x^{(i)})^2 = 2J(\theta)$

hence proved

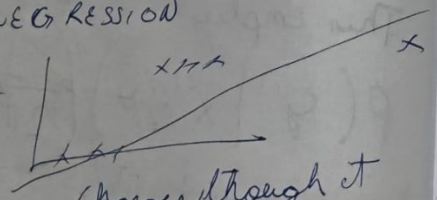
Classification Problem

NEVER USE LINEAR REGRESSION

ex:



but



Changes through it should not

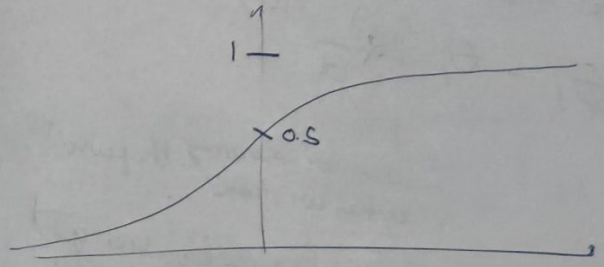
Logistic Regression

we want $h_\theta(x) \in [0,1]$

$$h_\theta(x) = g(\theta^T x) = \left(\frac{1}{1 + e^{-\theta^T x}}\right)$$

$\frac{1}{1+e^{-z}}$ is sigmoid / logistic function

graph:



in linear regression we had $\theta^T x$ output, we put it in sigmoid to force value b/w $0 \le 1$

non-convex

non-convex

$y = 1$ or $y = 0$

$$p(y=1|x;\theta) = h_\theta(x)$$

$$p(y=0|x;\theta) = 1 - h_\theta(x)$$

$$p(y|x;\theta) = (h_\theta(x))^{y} \cdot (1 - h_\theta(x))^{1-y}$$

now, $\because$ we know

$$L(\theta) = p(\vec{y}|x;\theta) = \prod_{i=1}^{m} p(y^{(i)}|x^{(i)};\theta)$$

$$= \prod_{i=1}^{m} h_\theta(x^{(i)})^{y^{(i)}} \cdot (1 - h_\theta(x^{(i)}))^{1-y^{(i)}}$$

$$l(\theta) = \log(L(\theta))$$

$$= \sum_{i=1}^{m} \left[y^{(i)} \log\left(h_\theta(x^{(i)})\right) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)})) \right]$$

now choose θ to maximize $l(\theta)$

for that we use batch gradient descent

$$\theta_j := \theta_j \{+\alpha\} \frac{\partial}{\partial \theta_j} l(\theta)$$

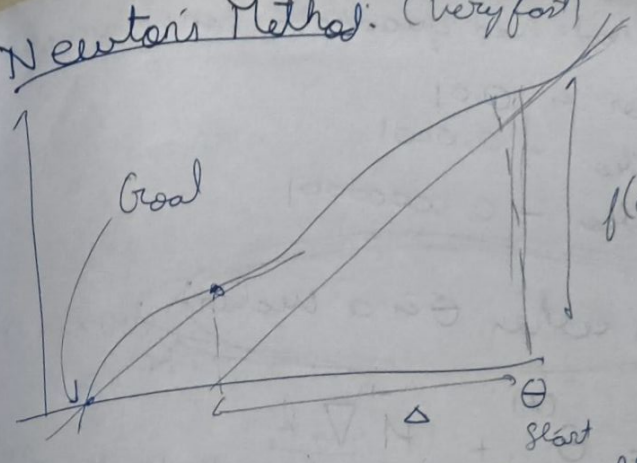
- when we minimize the function
+ when we max

after partial derivative we get

$$\theta_j := \theta_j + \alpha \left[\left(\sum_{i=1}^{m} (y^i - h_\theta(x^i)) \right) x_j^{(i)} \right]$$

this method is same as linear regression! The only difference is definition of $h_\theta(x)$

Newton's Method: (very fast)



we have $f'(\theta^\circ) = \frac{f(\theta^\circ)}{\Delta}$

$$\Rightarrow \Delta = \frac{f(\theta^\circ)}{f'(\theta^\circ)}$$

Newton's method is basically for we have f we want θ, s.t. $f(\theta) = 0$.

[$f(\theta)$ will be replaced by $l'(\theta)$ instead]
i.e derivative $= 0$

here we get:

$$\theta^{(t+1)} := \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}$$ → this is how we change θ

$$\theta := \theta - \frac{f(\theta)}{f'(\theta)}$$ old

let $f(\theta) = l'(\theta)$

$$\Rightarrow \theta^{(t+1)} := \theta^{(t)} - \frac{l'(\theta^{(t)})}{l''(\theta^{(t)})}$$

newton's method has quadratic convergence

so if error is 0.01
next iteration → 0.0001
next iteration → 0.00000001

Now when θ is a vector: $(n+1) \times (n+1)$ matrix

$$\theta^{(t+1)} := \theta^{(t)} + H^{-1} \nabla_\theta \ell$$

$\nabla_\theta \ell$ ← vector derivative $\mathbb{R}^{n+1}$

H is a hessian matrix

$$H_{ij} = \frac{\partial^2 \ell}{\partial \theta_i \partial \theta_j}$$

↑ clearly each iteration cost is alot

Lecture 4

Perceptron Algo

$$g(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

$$h_\theta(x) = g(\theta^T x)$$

update rule similar as last:

$$\theta_j := \theta_j + (\alpha \cdot x_j^{(i)})(y^{(i)} - h_\theta(x)^{(i)})$$

$y^{(i)} - h_\theta(x^{(i)}) = 0$ if algo guess correct
+/-1 if wrong

Exponential Families

PDF: probability distribution function

$$p(y;\eta) = b(y) \exp\left[\eta^T T(y) - a(\eta)\right]$$

y = data (we don't use x here for a reason)
η = natural parameter → dimension must match
$T(y)$ = Sufficient statistic
└ $T(y) = y$ for today

$b(y)$ = Base measure
$a(\eta)$ = log partition function

$$= \frac{b(y) \cdot \exp(\eta^T T(y))}{e^{a(\eta)}}$$ → kind of like normalizing term

we choose a, b, T?

Bernoulli distribution (Binary)

ϕ = probability of event

$$p(y_i; \phi) = \phi^{y_i}(1-\phi)^{1-y_i}$$ → PDF of bernoulli

now we want this form to be shoved into the above form and see what a, b, T turns out to be.

$$P(y;\phi) = \phi^y (1-\phi)^{(1-y)}$$

$$= \exp\left(\log\left(\phi^y (1-\phi)^{1-y}\right)\right)$$

$$= \exp\left(\underbrace{\log\left(\frac{\phi}{1-\phi}\right)}_{\eta} \cdot \underbrace{y}_{T(y)} + \underbrace{\log(1-\phi)}_{a(\eta)}\right)$$

so now $b(y) = 1$

$\eta = \log\left(\frac{\phi}{1-\phi}\right)$ → by matching pattern

$T(y) = y$

$a(\eta) = -\log(1-\phi)$

$= -\log\left(1 - \frac{1}{1+e^{-\eta}}\right) = \log(1+e^{\eta})$

$\Rightarrow \phi = \frac{1}{1+e^{-\eta}}$

Gaussian (with fixed variance)

Assume $\sigma^2 = 1$

$$P(y;\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y-\mu)^2}{2}\right)$$

PDF of Gaussian:

$$= \underbrace{\frac{1}{\sqrt{2\pi}} e^{\frac{-y^2}{2}}}_{b(y)} \exp\left(\underbrace{\mu}_{\eta} \cdot \underbrace{y}_{T(y)} - \underbrace{\frac{1}{2}\mu^2}_{a(\eta)}\right)$$

$\eta = \mu$

$T(y) = y$

$b(y) = \frac{e^{\frac{-y^2}{2}}}{\sqrt{2\pi}}$

$a(\eta) = \frac{\mu^2}{2} = \frac{\eta^2}{2}$

Real Values → Gaussian
Binary → Bernoulli
Count → Poisson
$\mathbb{R}^+$ → Gamma
Distribution → Beta
↳ Bayesian

we use exponential families because they have nice properties

(a) MLE (maximum likelihood) wrt η is concave
NLL (negative log likelihood) is ∴ convex

(b) expectation of y: $E[y;\eta] = \frac{\partial a(\eta)}{\partial \eta}$

(c) $\mathrm{Var}[y;\eta] = \frac{\partial^2}{\partial \eta^2} a(\eta)$ H.W prove these properties

General Linear Models (GLM)

Assumptions / design choices:

(i) $y|x;\theta \sim$ Exponential Family(η) ← η is a number of

(ii) $\eta = \theta^T x$, $\theta \in \mathbb{R}^n$, $x \in \mathbb{R}^n$ → n related

(iii) Test time → output $E[y|x;\theta]$

↳ $h_\theta(x) = E[y|x;\theta]$ the mean kind of like Bernoulli = logistic Reg, Gaussian = linear Reg.

$= E[y|\eta]$

Blueprint:

$x \longrightarrow$ [linear model $\theta^T x$] $\longrightarrow \eta \longrightarrow$ (End form in expo form with b, a, T and all)

so we train theta to predict η which is parameter for the distribution of which has mean as the output

so during training maximize $\log\left(P(y^{(i)}; \theta^T x^{(i)})\right)$

GLM Training

learning updation rule plays same:

$$\theta_j := \theta_j + \alpha\, x_j^{(i)} \left(y^{(i)} - h_\theta(x^{(i)})\right)$$

Σ for batch gradient

$E[y;\eta] = g(\eta)$ → canonical response function

$\eta = g^{-1}(\mu)$ → link function

(μ ?? y ?? → mean) $\mu = a(\eta)$

and as per properties $g(\eta) = \frac{\partial a(\eta)}{\partial \eta}$ the only learnable para

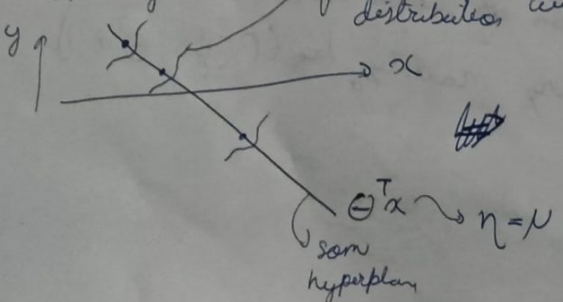
3 parametrization:
- model para → θ
- natural para → $\eta = \theta^T x$
- canonical para → ϕ ($\eta = g(\phi)$, $\phi = g^{-1}(\eta)$)

ϕ for bernoulli, λ for poisson, $\nu\sigma^2$ for Gaussian

example in logistic regression: $h_\theta(x) = E[y|x;\theta] = \phi = \frac{1}{1+e^{-\eta}}$

Assumptions

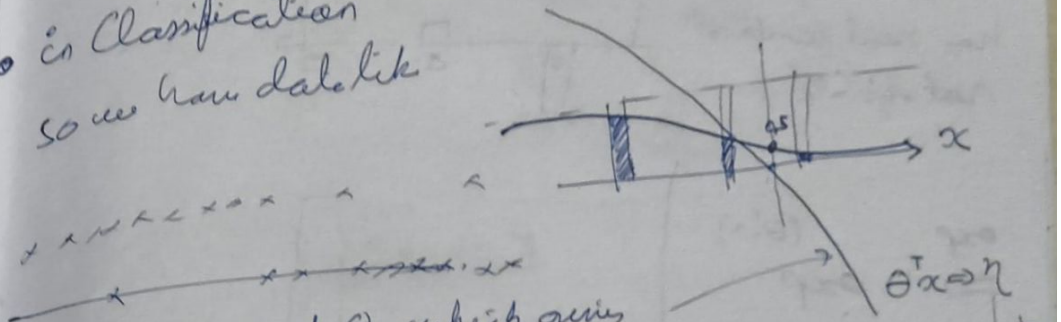
- in regression, for every possible x there is a gaussian distribution with $\sigma^2 = 1$ with η as mean



so the $\theta^T x$ is a line which represents the mean of y of x from which they are sampled

↳ maximum likelihood

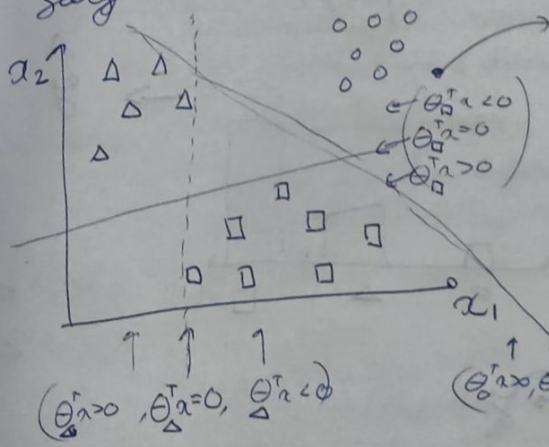
- in Classification so we have data like



now we want θ which gives

Softmax Regression → cross entropy minimisation

say we have 3 classes of data



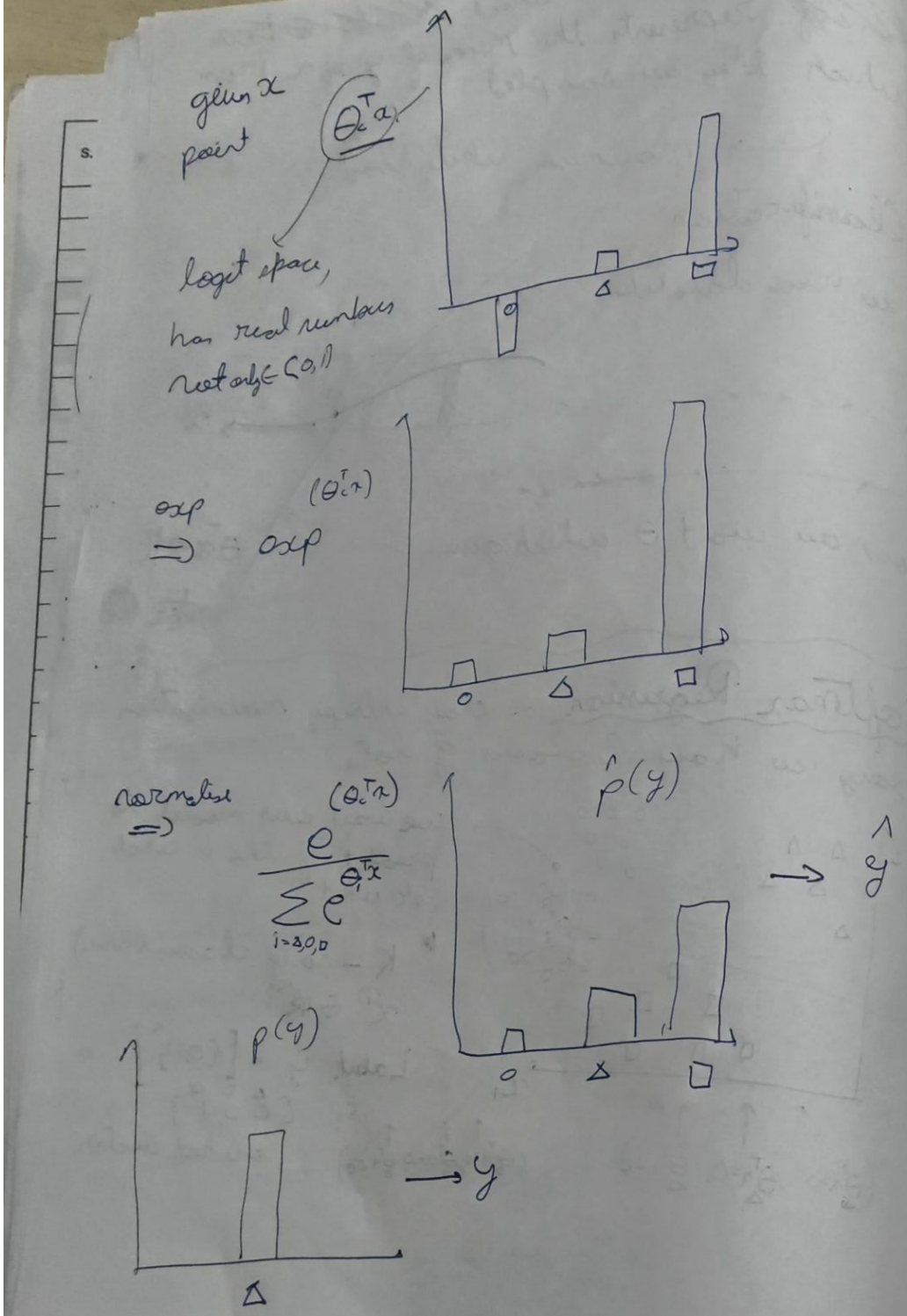
we want our model to predict if this is circle or what

- K – no. of classes (3 here)

$x^{(i)} \in \mathbb{R}^n$

Label $y = \begin{bmatrix} \{0,1\} \\ \vdots \\ k \end{bmatrix}$

Ex: $\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ one hot vector



now we want $\hat{p}$ to look like p, for this we minimize cross entropy

$$\text{Cross ent}(p,\hat{p}) = -\sum_{y \in (\circ,\Delta,\square)} p(y) \log \hat{p}(y)$$

$$= -\log \hat{p}(y_\Delta) \rightarrow \text{for this example}$$

$$= -\log \left(\frac{e^{\theta_\Delta^T x}}{\sum_{i=(\Delta,\square,\circ)} e^{\theta_i^T x}} \right)$$

Lecture 1-4

→ Introductions
- Supervised & unsupervised learning
- ↳ Regression and classification

*

→ Linear Regression
- hypothesis & θ update rule : gradient descent
- gradient descent : batch v/s stochastic
- normal equation & derivation → properties of trace deriv

→ Locally Weighted Regression
- weight function definition
- intro to distribution notations

→ Why we use least squares in linear regression (★)

→ Logistic Regression
- new hypothesis with same θ update rule
- newtons method for scalar θ
- newtons method for vector θ (★)

→ Perceptron Algo
→ Exponential Families
→ General Linear Models
- definition, blueprint & training

→ Softmax Regression (★)